

Runtime Analysis of Hybrid Sort

Theoretical and Empirical Considerations for Hyperparameter Tuning

D. Tan, H. G. De Vera, P. Francis

Contents

1. Theoretical Framework (Runtime)
 - Time Complexity Analysis
 - Modeling the Runtime
2. Theoretical Framework (Key Comparisons)
 - Worst-Case Key Comparisons
 - Optimizing the Worst-Case Number of Key Comparisons
 - Summary of Analysis
3. Algorithm Implementation

- Algorithm Implementation (hybrid_merge(a) Skeleton)
- Algorithm Implementation (Insertion Sort Branch)
- Algorithm Implementation (Merge Sort Branch Initialization)
- Algorithm Implementation (Merge Sort Branch, Merging)
- Algorithm Implementation (Merge Sort Branch, Writing Tail)

4. Input Data Generation

- Input Data Generation

5. Theoretical Results

- Approximations of the Worst-Case Key Comparisons
- Varying the Array Size
- Varying the Insertion Sort Threshold

6. Empirical Results

- Testing Methodology
- Varying the Array Size
- Varying the Insertion Sort Threshold
- Varying Both n and s to Determine Optimal s
- Regression Analysis to Determine Optimal s
- Comparison of Hybrid Sort With Merge Sort

7. Appendix A: Worst-Case Analysis of the Number of Key Comparisons

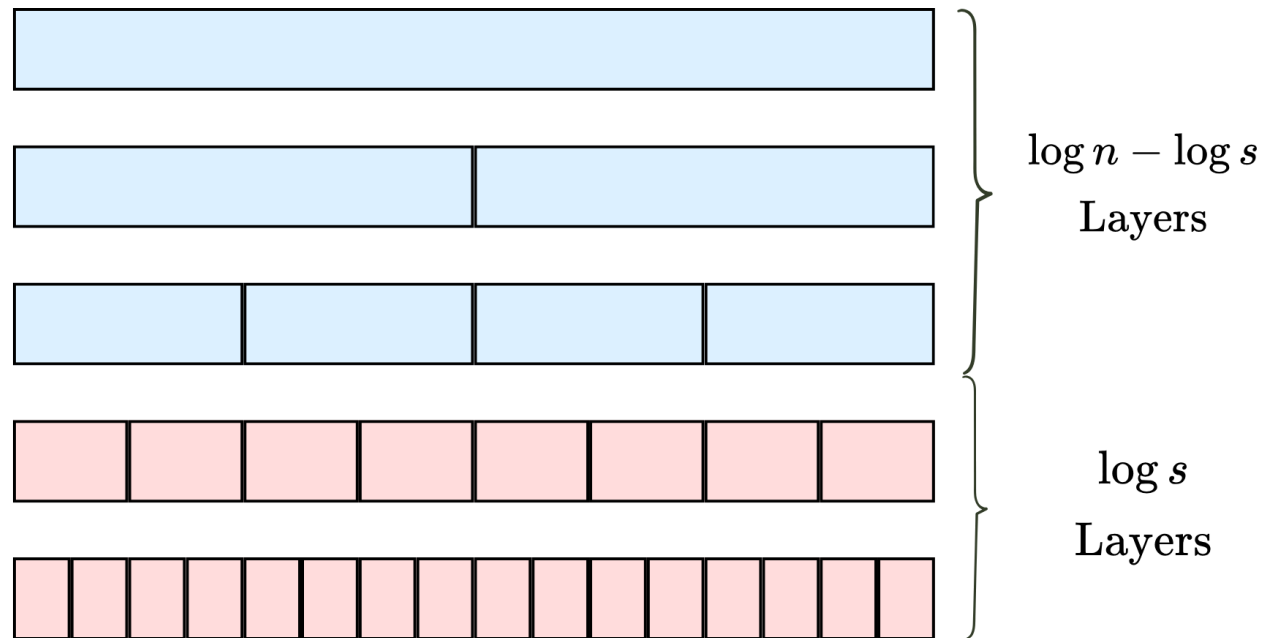
8. Appendix B: Co-optimality of $s = 1, 2, 3$

9. Appendix C: Computing R^2

10. [Appendix D: Computing Optimal \$s\$ Using Centre of Mass](#)
11. [Appendix E: Detailed Time Complexity Analysis of Hybrid Sort](#)
12. [Appendix F: Modeling the Runtime](#)
13. [Appendix G: Regression Methodology](#)

Theoretical Framework (Runtime)

- Hybrid sort subdivides the array into smaller blocks. For blocks of size s or smaller, it uses insertion sort. For larger blocks, it uses merge sort.
- Let s be the **insertion sort threshold**, the max size of the subarray where we use insertion sort. There are $O(\frac{n}{s})$ blocks that we must sort with insertion sort, and each block takes $O(s^2)$ time to sort, hence $O(ns)$ time spent sorting small blocks.

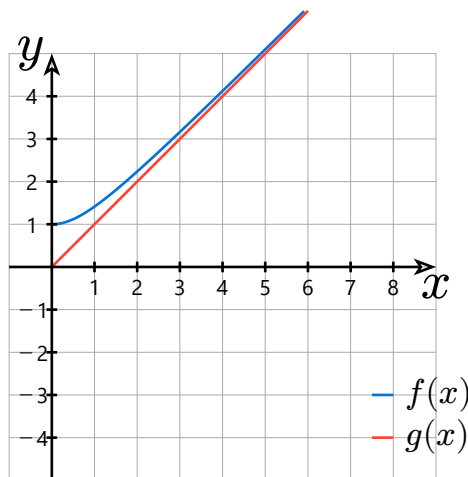


- See Appendix E for a more detailed analysis.

- The total time complexity is thus $O(n(s - \log s) + n \log n) = O(ns + n \log n)$.
- For fixed s , this simplifies to $O(n \log n)$.
- Hence it seems minimizing $s - \log s$ suffices.
- Attempting to minimize $n(s - \log s) + n \log n$ yields $s = 1$.

- For minimizing key comparisons, this analysis matches experimental results.
- For minimizing process runtime, this approach fails.
- **Problem:** Big-O hides relevant constant factors in the objective function.

Idea: Introduce Constants



- **Definition:** Two functions f, g are asymptotically equivalent iff $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1$
- **Assume:**
 - Runtime of insertion sort on n elements is asymptotically equivalent to $f(n) = an^2$ for some $a \in \mathbb{R}$.
 - Runtime of merge sort on n elements is asymptotically equivalent to $g(n) = bn \ln n$ for some $b \in \mathbb{R}$.

New Objective: Minimize total runtime¹

$$F(n, s) \approx ans + bn \log n - bn \log s$$

- Under this model, **F is minimized at $s = \frac{b}{a}$ independent of n** . The derivation involves calculus (see Appendix F).

¹We assume $\log$ is the natural log for ease of analysis. It makes the calculus cleaner. The choice of base is irrelevant as it is absorbed within the constants a, b .

Optimal s to minimize runtime:

$$s = \frac{b}{a}$$

- Hence, as the constant of **merge sort** (b) **increases**, the model recommends **increasing** s so as to spend less time in merge sort and more time in insertion sort.
- Meanwhile, as the constant of **insertion sort** (a) **increases**, the model recommends **decreasing** s to reduce time spent in insertion sort.
- Model passes sanity check. 
- How to find a, b ? **Regression.**

Theoretical Framework (Key Comparisons)

- Let $T(n, s)$ be the **worst-case number of key comparisons** produced by Hybrid Sort over all arrays of size n , assuming an **insertion sort threshold** of s .
- We have the following **exact recurrence** for $T(n, s)$:

$$T(n, s) = \begin{cases} n - 1 + T(\lfloor \frac{n}{2} \rfloor, s) + T(\lceil \frac{n}{2} \rceil, s) & \text{if } n > s \\ \frac{n(n-1)}{2} & \text{otherwise} \end{cases}$$

- The above is computable in $O(\log_2(\frac{n}{s}))$ time for fixed n and s .

- We have both **heuristic evidence** (Appendix A) and a **mathematical induction proof** (Appendix B) that $s = 1$, $s = 2$, and $s = 3$ are the only s that minimize the **worst-case number of key comparisons for all n** .
- In the interest of time, we will skip the proof.

Objective	Theoretical Optimal s	Machine-Dependent?
Minimize Runtime	$s = \frac{b}{a}$	Yes
Minimize Worst-case Key Comparisons	$s = 1, 2, 3$	No

Algorithm Implementation

```
s = 10
keycomp = 0
def hybrid_merge(a):
    global keycomp, s
    if(len(a) <= s):
        # Insertion Sort
    else:
        # Merge Sort
    return a
```

```
for i in range(1, len(a)):
    for j in range(i - 1, -1, -1):
        keycomp += 1
        if(a[j] > a[j + 1]):
            a[j], a[j + 1] = a[j + 1], a[j]
        else:
            break
```

```
n = len(a)
m = n >> 1
l = a[0:m]
r = a[m:n]
l = hybrid_merge(l)
r = hybrid_merge(r)
l_size = m
r_size = n - m
i = 0
j = 0
ptr = 0
```

```
while(i < l_size and j < r_size):  
    keycomp += 1  
    if(l[i] <= r[j]):  
        a[ptr] = l[i]  
        i += 1  
    else:  
        a[ptr] = r[j]  
        j += 1  
    ptr += 1
```

```
while(i < ls):  
    a[ptr] = l[i]  
    i += 1  
    ptr += 1  
while(j < rs):  
    a[ptr] = r[j]  
    j += 1  
    ptr += 1
```

Input Data Generation

```
from random import randint

# largest integer possible in data set
x = 10**18

def gen_random_list(n):
    return list(map(lambda _: randint(1, x), range(n)))
```

Theoretical Results

- By applying some approximations and recurrence unrolling (see Appendix A), we obtain the following approximation for $T(n, s)$:

$$T(n, s) \approx f_1(n, s) := n \left(\log_2 \left(\frac{n}{s} \right) + \log_2(3) - 1 \right) - \frac{3n}{2s} + 1 + n \left(\frac{s}{3} - \frac{1}{2} \right)$$

- Notice that $f_1(n, s) = \Theta(ns + n \log n)$, which agrees with the theoretical.
- We also found the following—better—approximation:

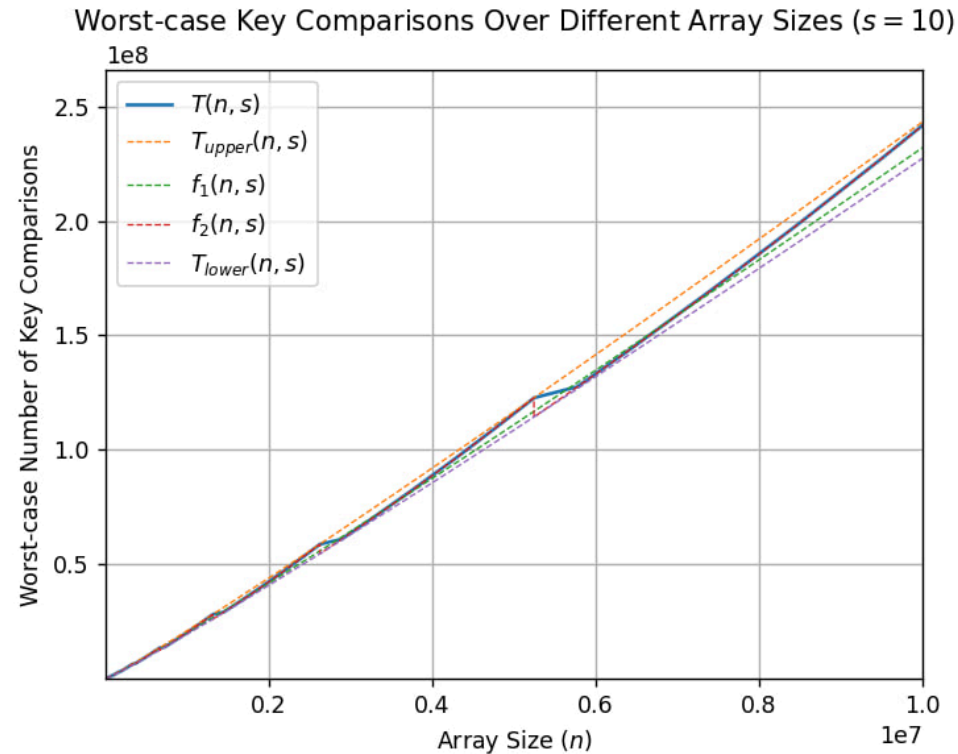
$$T(n, s) \approx f_2(n, s) \\ := n \left(\left\lceil \log_2 \left(\frac{n}{s} \right) \right\rceil - 2^{\lceil \log_2(\frac{n}{s}) \rceil} \right) - 2 \left\lceil \log_2 \left(\frac{n}{s} \right) \right\rceil + 1 + n \left(\frac{s}{2^{\lceil \log_2(\frac{n}{s}) \rceil - \log_2(\frac{n}{s}) + 1}} - \frac{1}{2} \right)$$

- Meanwhile, we found the following approximate upper and lower bounds:

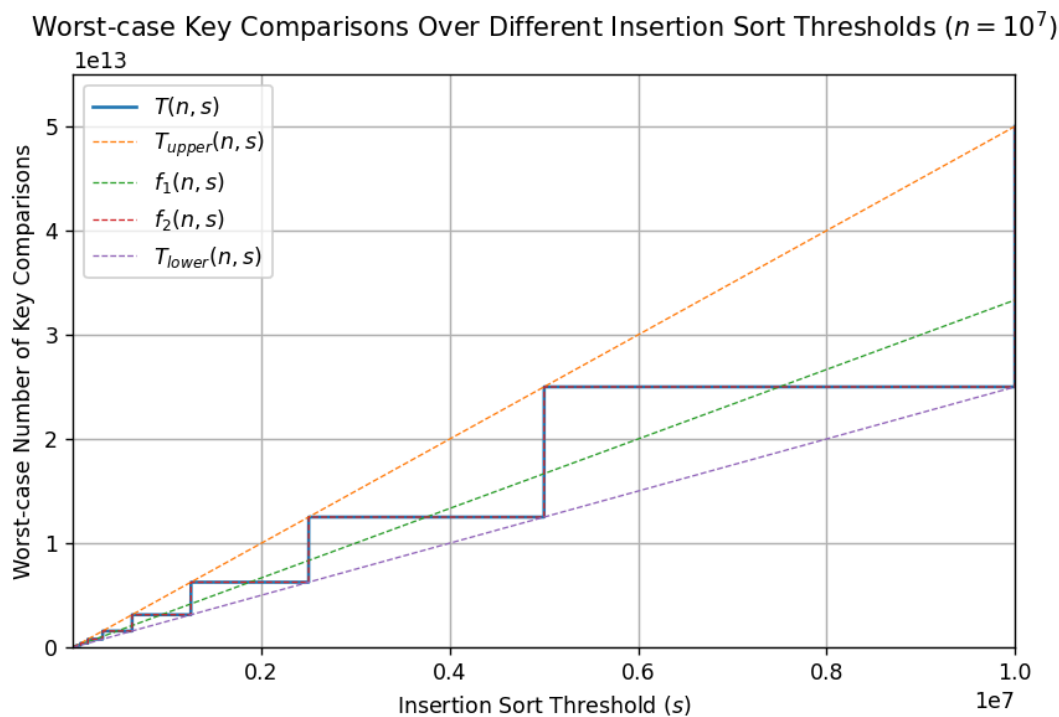
$$T_{\text{upper}}(n, s) := n \log_2 \left(\frac{n}{s} \right) - \frac{n}{s} + 1 + n \left(\frac{s}{2} - \frac{1}{2} \right) = \Theta(ns + n \log n)$$

$$T_{\text{lower}}(n, s) := n \left(\log_2 \left(\frac{n}{s} \right) + 1 \right) - \frac{2n}{s} + 1 + n \left(\frac{s}{4} - \frac{1}{2} \right) = \Theta(ns + n \log n)$$

- Hence, we have further evidence that hybrid sort runs in $O(ns + n \log n)$.



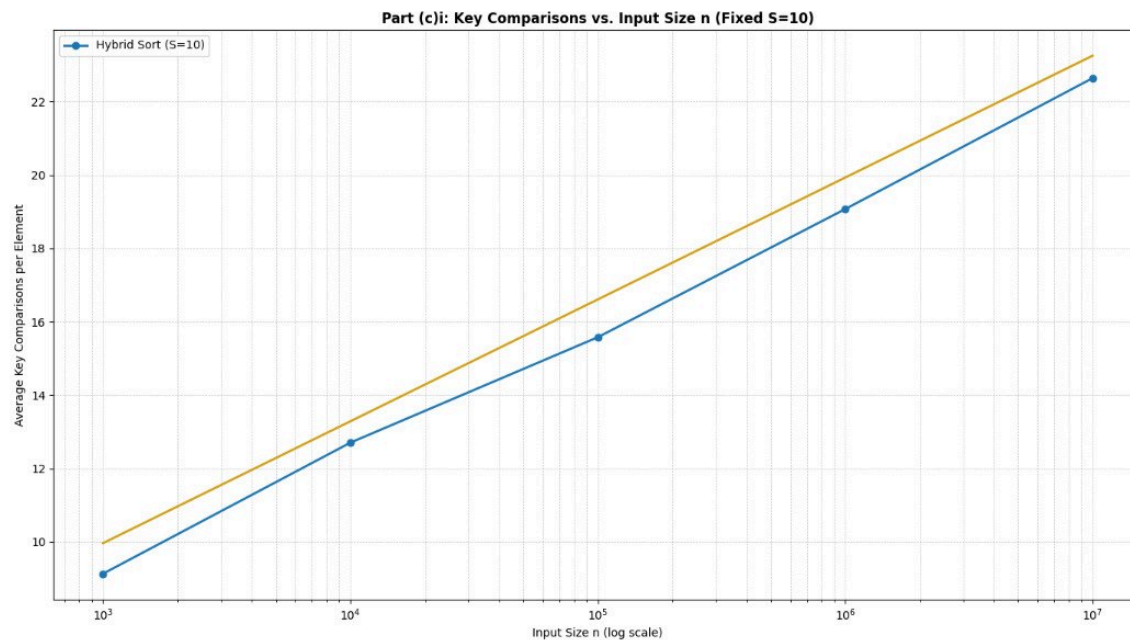
- $f_1(n, s)$ (in green) achieves $R^2 = 0.9978$ for $s = 10$ and varied n
- $f_2(n, s)$ (in red) achieves $R^2 = 0.9997$ for $s = 10$ and varied n



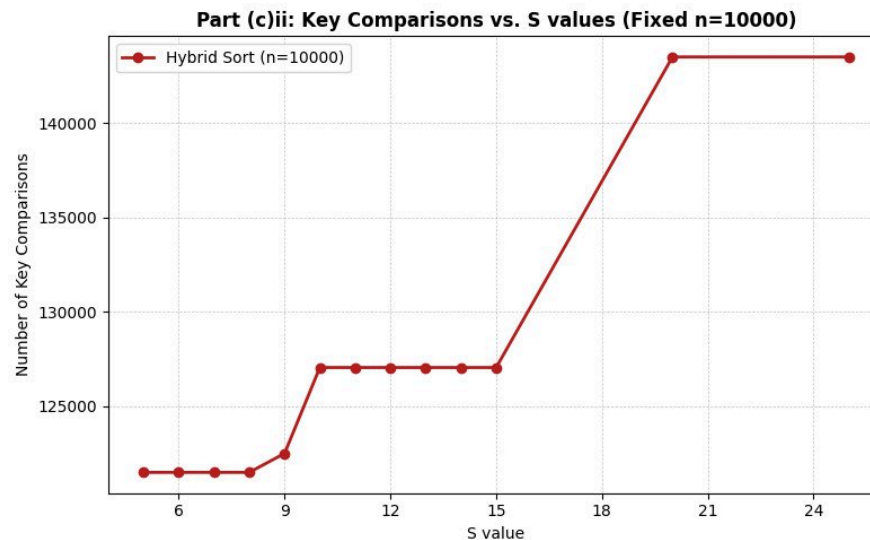
- $f_1(n, s)$ (in green) achieves $R^2 = 0.8333212567988$ for $n = 10^7$ and varied s
- $f_2(n, s)$ (in red) achieves $R^2 = 0.9999999999995$ for $n = 10^7$ and varied s

Empirical Results

- Timing was done with `time.perf_counter` in Python as this timer seems to minimize variance out of the available built-in timers.
- In randomized test cases, we performed multiple timing measurements over different inputs and averaged across them. This aims to **lessen the influence of confounding variables**.
- We ran all tests on a single computer to eliminate runtime and environment differences.



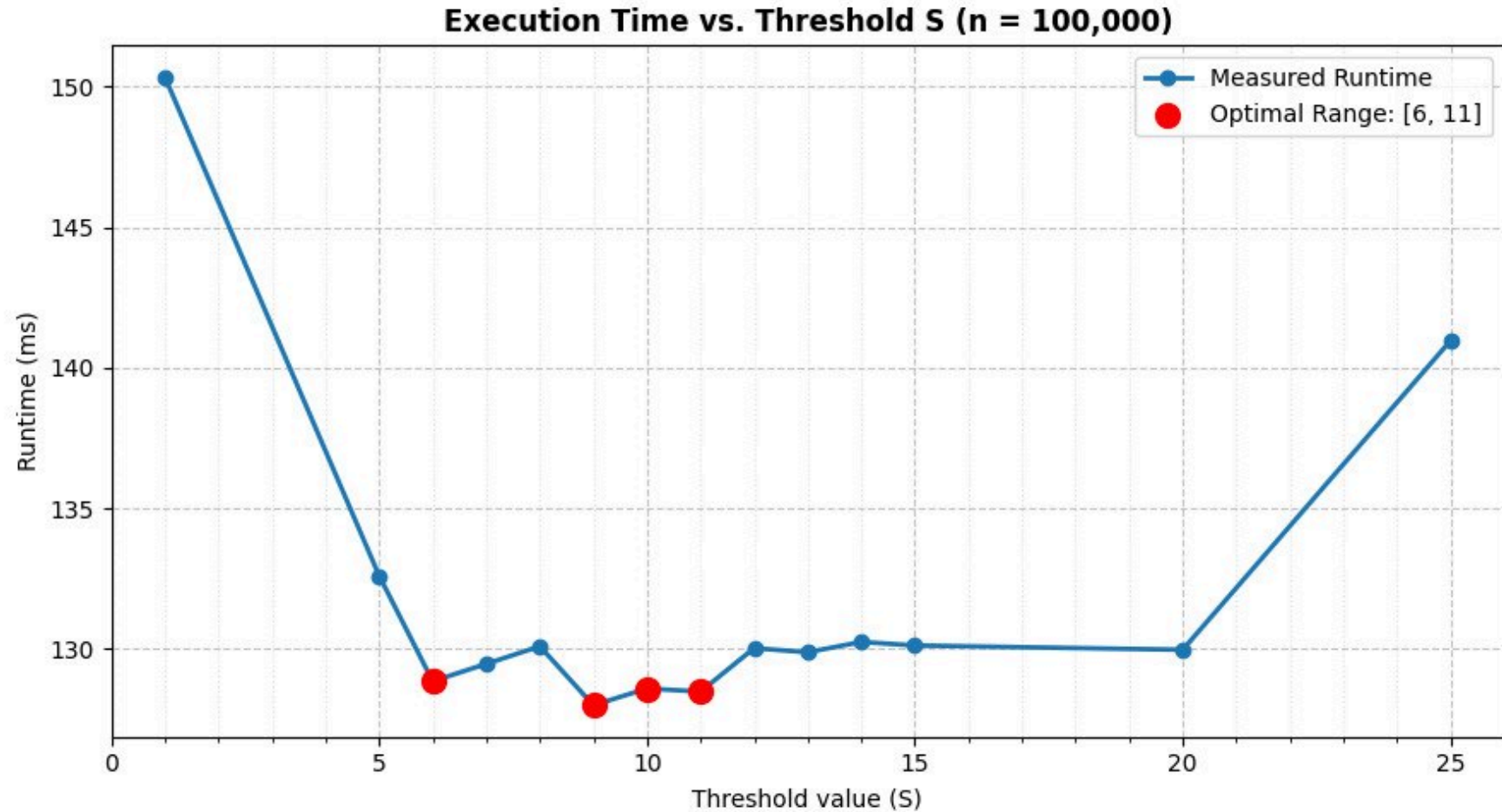
- We plotted the average key comparisons divided by n against varying input sizes.
- Confirms theoretical analysis that if s is fixed, runtime is $O(n \log n)$.

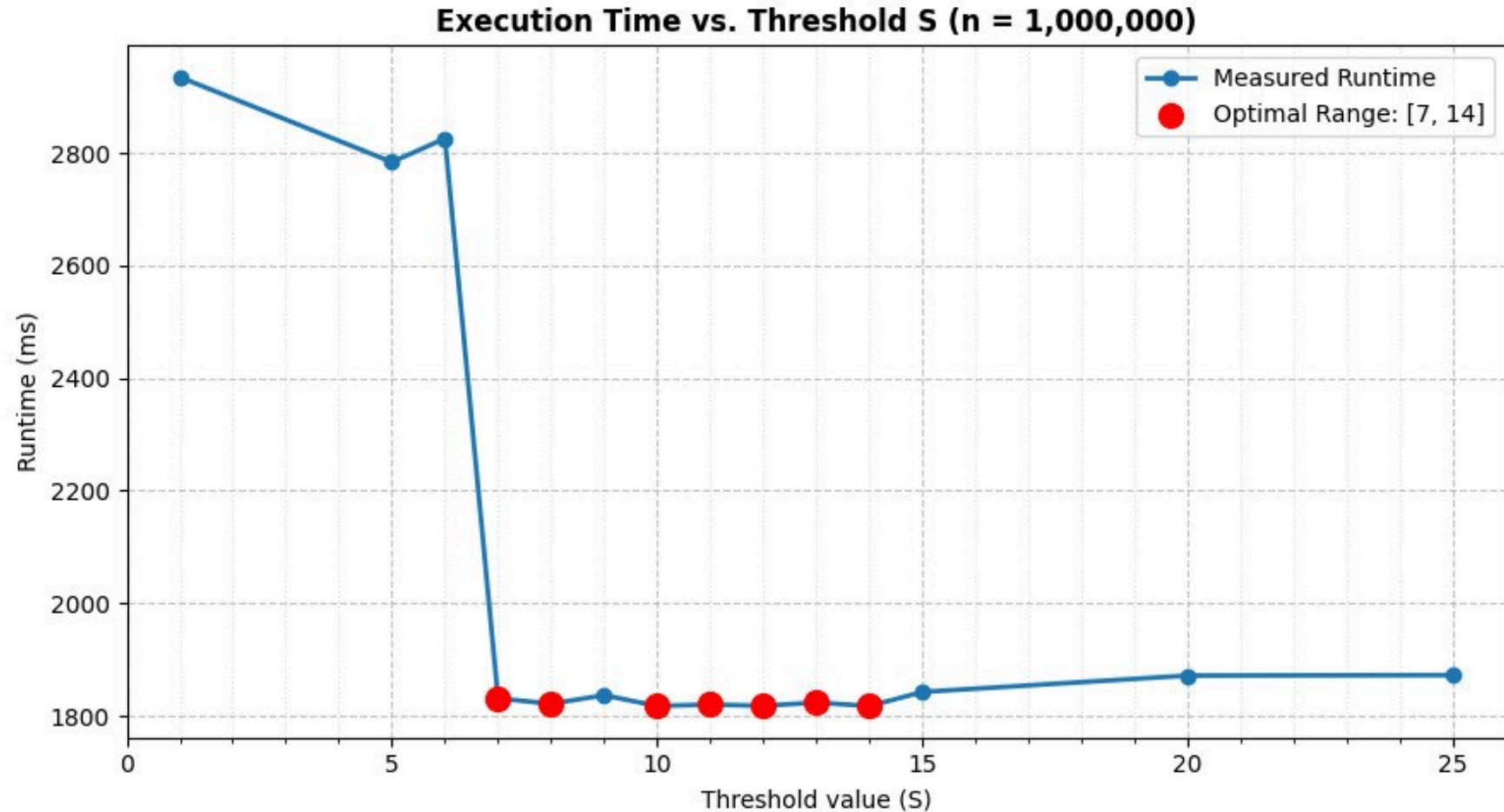


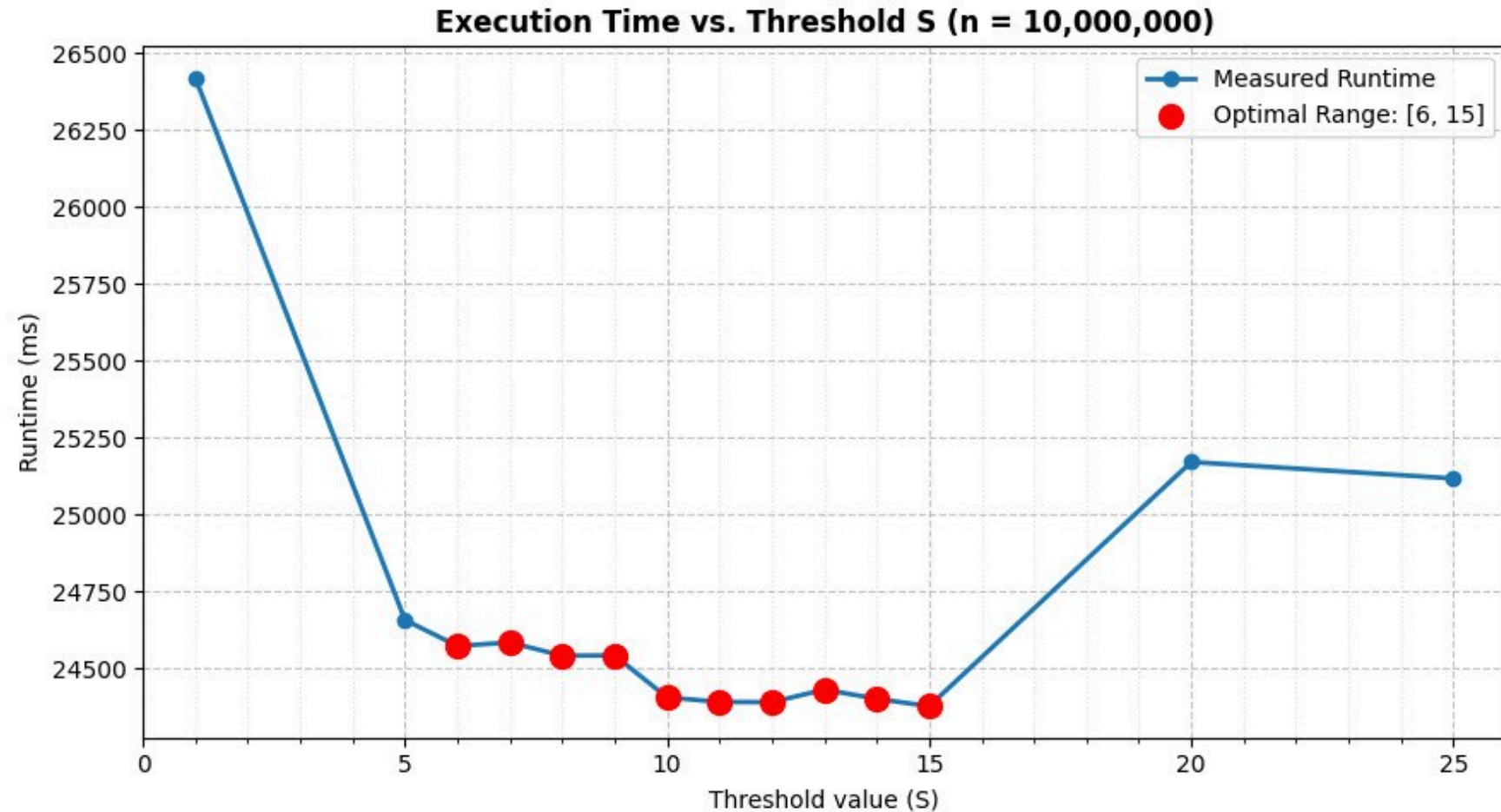
- The above shows **avg. number of key comparisons** for $n = 10000$ and varying s .
- We notice that the graph tends to plateau at certain s values, then jump at certain thresholds. We found that these jumps occur when $s \approx \frac{n}{2^k}$ for some $k \in \mathbb{Z}$.
- We notice that the number of key comparisons is minimized at $s = 1, 2, 3$, consistent with our theoretical model.

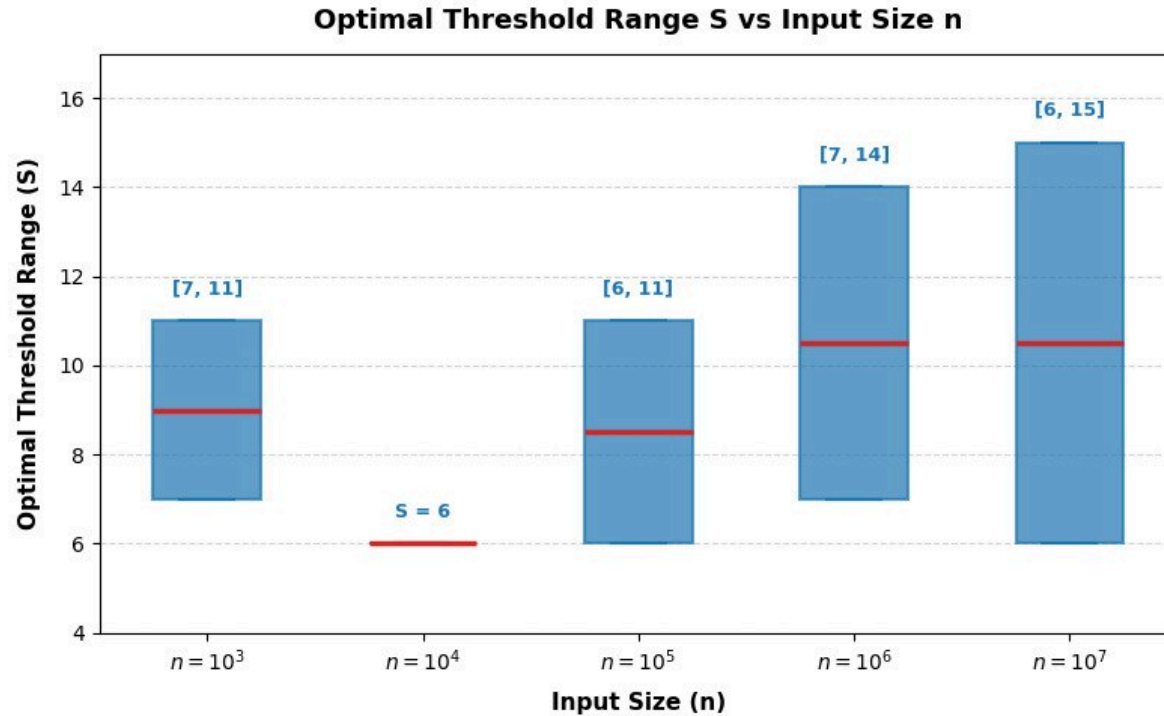












- The optimal s value (by center of mass) is $s \approx 9.7 \approx 10$.

Runtime Model

$$F(n, s) \approx ans + bn \log n - bn \log s$$

- We apply **least-squares regression** on our running time data across different values of (n, s) using the model F . (See Appendix G)
- We obtain $a \approx 1.61 \cdot 10^{-5}$, $b \approx 1.65 \cdot 10^{-4}$, and $R^2 = 0.9996$, indicating a strong correlation between our model and the data.
- Using $s = \frac{b}{a}$, we obtain the following as the theoretical optimal value of s :

$$s_{\text{opt}} = \frac{b}{a} \approx 10.2 \approx 10$$

- Our theoretical model matches our empirical analysis!

Final Verdict (Performance Comparison for $n = 10^7$)

s	Average Runtime (s)	Average Key Comparisons
1 (Normal Merge Sort)	26.420	$2.201 \cdot 10^8$
10 (Hybrid Sort)	24.406	$2.264 \cdot 10^8$

- We observe a **7.6%** increase in performance, with only a **2.9%** increase in the average number of key comparisons when tested for $n = 10^7$.

Thanks for Listening!

Appendix A: Worst-Case Analysis of the Number of Key Comparisons

- Assume **0-based indexing** for this analysis.
- For insertion sort, the worst-case is that the array is reverse-sorted. In this case, we would have to move the i th element to the very beginning, incurring i swaps. In total, this gives $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$ swaps for an array of size n .

- For merge sort, all key comparisons occur during the merge step.
- Let $T(n)$ be the worst-case number of comparisons for an array of size n .
- Supposing we split the array into $[0, \lfloor \frac{n}{2} \rfloor - 1]$, $[\lfloor \frac{n}{2} \rfloor, n - 1]$, the worst-case is that, as we do the merge, the minimum element alternates between the left and right subarrays.
- In this case, we need to perform $n - 1$ key comparisons.
- This yields the recurrence $T(n) = n - 1 + T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil)$, where T is the worst-case number of key comparisons.

- For Hybrid Sort, we may modify the recurrence as follows:

$$T(n) = \begin{cases} n - 1 + T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) & \text{if } n > s \\ \frac{n(n-1)}{2} & \text{otherwise} \end{cases}$$

- One can compute this in $O(\log n)$ time. Unfortunately, as currently formulated, we could not find a closed-form for this recurrence.
- Hence, we settled for an approximation:

$$T(n) \approx \begin{cases} n - 1 + 2T(\frac{n}{2}) & \text{if } n > s \\ \frac{n(n-1)}{2} & \text{otherwise} \end{cases}$$

- Note: $T(n)$ is not monotonically increasing, so we cannot cleanly bound it above.

- Let k be the smallest nonnegative integer so that $\frac{n}{2^k} \leq s$. Hence, $2^k \geq \frac{n}{s} \leftrightarrow k \geq \log_2\left(\frac{n}{s}\right)$
- Thus, $k = \lceil \log_2\left(\frac{n}{s}\right) \rceil$. We can then unroll the recurrence, giving us the approximate formula:

$$T(n) \approx nk - 2^k + 1 + \frac{n(n - 2^k)}{2^{k+1}}$$

- As $2^k \approx 1.5 \frac{n}{s}$, $k \approx \log_2\left(\frac{n}{s}\right) + \log_2(3) - 1$.
- Hence, an approximation for $T(n)$ is:

$$T(n) \approx n \left(\log_2\left(\frac{n}{s}\right) + \log_2(3) - 1 \right) - \frac{3n}{2s} + 1 + \frac{n \left(n - \frac{3n}{2s} \right)}{\frac{3n}{s}}$$

- Simplifying:

$$T(n) \approx n \left(\log_2 \left(\frac{n}{s} \right) + \log_2(3) - 1 \right) - \frac{3n}{2s} + 1 + n \left(\frac{s}{3} - \frac{1}{2} \right)$$

- Let $f(n, s) = n \left(\log_2 \left(\frac{n}{s} \right) + \log_2(3) - 1 \right) - \frac{3n}{2s} + 1 + n \left(\frac{s}{3} - \frac{1}{2} \right)$. We have:

$$\frac{\partial f}{\partial s}(n, s) = -\frac{n}{s \ln(2)} + \frac{3n}{2s^2} + \frac{n}{3}$$

- Notice:

$$\frac{s^2}{3} - \frac{s}{\ln(2)} + \frac{3}{2} = 0 \leftrightarrow -\frac{1}{s \ln(2)} + \frac{3}{2s^2} + \frac{1}{3} = 0 \leftrightarrow \frac{\partial f}{\partial s}(n, s) = -\frac{n}{s \ln(2)} + \frac{3n}{2s^2} + \frac{n}{3} = 0$$

- Thus, $\frac{\partial f}{\partial s}(n, s) = 0$ iff $s \approx 1.73616$ or $s \approx 2.59192$. In fact, f is minimized at $s \approx 1.72795$. However, as s must be an integer, f is minimized either at $s = 1$, $s = 2$, or $s = 3$.
- Hence, as $T(n) = f(n, s)$, we find that $T(n)$ is likely minimized at $s = 1$, $s = 2$, or $s = 3$.
- This is heuristic evidence that the minimum number occurs at around $s = 1, 2, 3$.
- This is consistent with our empirical results!
- The difference between minimizing runtime and minimizing number of key comparisons is that the number of key comparisons is implementation-independent. This is why it is minimized at a fixed constant, independent of constant factors a, b .

Appendix B: Co-optimality of $s =$ 1, 2, 3

- The heuristic argument in Appendix A, as well as the empirical results, motivates us to find an mathematical proof of optimality.
- We employ a technique known as **strong induction** to demonstrate the co-optimality of $s = 1$, $s = 2$, and $s = 3$ under the objective of minimizing the number of key comparisons done.

- Let $T(n, s)$ be the worst-case number of key comparisons of hybrid sort on an array with n elements and an insertion sort threshold of at most s elements.
- Recall from Appendix A that:

$$T(n, s) = \begin{cases} n - 1 + T(\lfloor \frac{n}{2} \rfloor, s) + T(\lceil \frac{n}{2} \rceil, s) & \text{if } n > s \\ \frac{n(n-1)}{2} & \text{if } n \leq s \end{cases}$$

- We wish to show, for all integers $s \geq 1$ and for all $n \geq 1$, that $T(n, s) \leq T(n, s + 1)$.
- Let s be an arbitrary integer ≥ 1 . We proceed by strong induction on n .
- If $1 \leq n \leq s : T(n, s) = T(n, s + 1) = \frac{n(n-1)}{2}$, so $T(n, s) \leq T(n, s + 1)$.

- If $n = s + 1$: $T(n, s + 1) = \frac{s(s+1)}{2}$ and $T(n, s) = s + T(\lfloor \frac{s+1}{2} \rfloor, s) + T(\lceil \frac{s+1}{2} \rceil, s)$.
- We note that $f(x) = \frac{x(x-1)}{2}$ is nondecreasing over $x \geq 0$. Hence, $x \geq y \geq 0 \rightarrow f(x) \geq f(y)$.

- Case 1: $s \geq 4$
- Since $\lfloor \frac{s+1}{2} \rfloor \leq \lceil \frac{s+1}{2} \rceil \leq s$ for $s \geq 1$,

$$T(n, s) = s + \frac{\lfloor \frac{s+1}{2} \rfloor (\lfloor \frac{s+1}{2} \rfloor - 1)}{2} + \frac{\lceil \frac{s+1}{2} \rceil (\lceil \frac{s+1}{2} \rceil - 1)}{2}$$

$$\leq s + (\lceil \frac{s+1}{2} \rceil (\lceil \frac{s+1}{2} \rceil - 1))$$

$$\leq s + (\frac{s+2}{2})(\frac{s+2}{2} - 1)$$

$$= (\frac{s}{2} + 3)(\frac{s}{2})$$

$$\leq (s + 1)(\frac{s}{2}) \text{ (valid for } s \geq 4)$$

$$= \frac{s(s+1)}{2}$$

$$= T(n, s + 1)$$

- Hence, $T(n, s) \leq T(n, s + 1)$ for $n = s + 1$.

- Case 2: $s \leq 3$
- We also have $T(2, 1) = 1 \leq 1 = T(2, 2)$; and,
- $T(3, 2) = 3 \leq 3 = T(3, 3)$; and,
- $T(4, 3) = 5 \leq 6 = T(4, 4)$
- Hence, $T(n, s) \leq T(n, s + 1)$ for $n = s + 1$ and $s \leq 3$.
- As we have exhausted all cases, $T(n, s) \leq T(n, s + 1)$ for all integers $s \geq 1$.

- Finally, let n be an arbitrary integer $> s + 1$. Suppose $T(x, s) \leq T(x, s + 1)$ for $1 \leq x < n$. As $n > s + 1$, $n \geq s + 2 \geq 1 + 2 = 3$. Hence, $\lfloor \frac{n}{2} \rfloor \leq \lceil \frac{n}{2} \rceil < n$.

- Thus,

$$\begin{aligned} T(n, s) &= n - 1 + T(\lfloor \frac{n}{2} \rfloor, s) + T(\lceil \frac{n}{2} \rceil, s) \\ &\leq n - 1 + T(\lfloor \frac{n}{2} \rfloor, s + 1) + T(\lceil \frac{n}{2} \rceil, s + 1) \text{ (by the inductive hypothesis)} \\ &= T(n, s + 1) \end{aligned}$$

- Hence, $T(n, s) \leq T(n, s + 1)$.
- By the principle of strong mathematical induction, $T(n, s) \leq T(n, s + 1)$ holds for all n and for all s . $\square$

- We wish to show $T(n, 1) = T(n, 2) = T(n, 3)$ for all integers $n \geq 1$.
- First, we have $T(1, s) = 0$, $T(2, s) = 1$, $T(3, s) = 3$, and $T(4, s) = 5$ for $s \leq 3$.
- Hence we have proven the statement for $n \leq 4$.
- We proceed once again with strong mathematical induction.

- Let n be an arbitrary integer > 4 . Suppose $T(x, 1) = T(x, 2) = T(x, 3)$ for $1 \leq x < n$. As $n > 4$, $n \geq 5$.
- Hence, $\lfloor \frac{n}{2} \rfloor \leq \lceil \frac{n}{2} \rceil < n$.
- Further, as $s \leq 3$, $n > s + 1$.
- Thus,

$$\begin{aligned} T(n, 1) &= n - 1 + T(\lfloor \frac{n}{2} \rfloor, 1) + T(\lceil \frac{n}{2} \rceil, 1) \\ &= n - 1 + T(\lfloor \frac{n}{2} \rfloor, 2) + T(\lceil \frac{n}{2} \rceil, 2) \text{ (by the inductive hypothesis)} \\ &= T(n, 2) \\ &= n - 1 + T(\lfloor \frac{n}{2} \rfloor, 2) + T(\lceil \frac{n}{2} \rceil, 2) \\ &= n - 1 + T(\lfloor \frac{n}{2} \rfloor, 3) + T(\lceil \frac{n}{2} \rceil, 3) \text{ (by the inductive hypothesis)} \\ &= T(n, 3) \end{aligned}$$

- Hence, $T(n, 1) = T(n, 2) = T(n, 3)$.
- By the principle of strong mathematical induction, $T(n, 1) = T(n, 2) = T(n, 3)$ holds for all n . $\square$
- Hence, indeed, $s = 1$, $s = 2$, and $s = 3$ all attain the optimal worst-case number of comparisons.
- Further, as $T(4, 3) = 5 < 6 = T(4, 4) \leq T(4, s)$ for $s \geq 4$, $T(4, 3) < T(4, s)$ for $s \geq 4$.
- Hence, none of $s \geq 4$ could be optimal.
- Hence, $s = 1, 2, 3$ are the only s that achieve optimality. $\blacksquare$

Appendix C: Computing R^2

- Where applicable, we compute the coefficient of determination, R^2 , of a model (approximation) Y_{pred} with respect to ground truth (the actual data) Y as:

$$R^2 := 1 - \frac{SS_{\text{resid}}}{SS_{\text{tot}}} = 1 - \frac{\mathbb{E}[(Y - Y_{\text{pred}})^2]}{\text{Var}(Y)}$$

- The statistic $R^2 = a$ could be interpreted as “ Y_{pred} explains $(100a)\%$ of the variance in Y ”.
- The closer R^2 is to 1, the better the fit.

Appendix D: Computing Optimal s Using Centre of Mass

- Let $f(n, s)$ be the computed average runtime among randomly shuffled arrays of size n , with insertion sort threshold s .
- Let $T \in \mathbb{R}^+$ be the closeness acceptance threshold—which we set to $T = 0.01$.
- Let $\text{Opt}(n) := \left\{ \frac{|f(n, s) - \min_{s^* \in \mathbb{N}} f(n, s^*)|}{\min_{s^* \in \mathbb{N}} f(n, s^*)} \leq T \mid s \in \mathbb{N} \right\}$.
- We test the algorithm over $n \in N_{\text{tests}}$, where $N_{\text{tests}} := \{10^3, 10^4, 10^5, 10^6, 10^7\}$.
- Finally, we compute the centre of mass as:

$$\bar{s} = \frac{\sum_{n \in N_{\text{tests}}} \left(\frac{[\max \text{Opt}(n)][1 + \max \text{Opt}(n)]}{2} - \frac{[\min \text{Opt}(n)][-1 + \min \text{Opt}(n)]}{2} \right)}{\sum_{n \in N_{\text{tests}}} ([\max \text{Opt}(n)] - [\min \text{Opt}(n)] + 1)}$$

Appendix E: Detailed Time Complexity Analysis of Hybrid Sort

- To sort large blocks, we need to sort blocks whose sizes are in the interval $(s, n]$. This is exactly the work needed to sort the blocks whose sizes are in the interval $[1, n]$, minus the work needed to sort blocks of size $[1, s]$. Hence, $O(n \log n - n \log s)$ time spent sorting large blocks.

Appendix F: Modeling the Runtime

- **Model Limitation:** Assumes fixed constants a, b . Variance in runtime may occur due to hardware differences and specific implementation details.
- **Justification:** Since we run merge sort for large enough n and insertion sort on multiple instances, variations from the asymptotic behavior should tend to disappear.

New Objective: Minimize total runtime

$$F(n, s) \approx ans + bn \log n - bn \log s$$

- **Insight:** $F(n, s) \approx ans + bn \log n - bn \log s = bn \log n + n(as - b \log s)$
 - Hence equivalent to minimizing $as - b \log s$.
- Let $f(s) = as - b \log s$. Thus, $f'(s) = a - \frac{b}{s}$, which is zero when $s = \frac{b}{a}$.²
- Hence, f is minimized at $s = \frac{b}{a}$.
- Hence, under this model, **F is also minimized at $s = \frac{b}{a}$ independent of n .**

²We assume $\log$ is the natural log for ease of analysis. It makes the calculus cleaner. The choice of base is irrelevant as it is absorbed within the constants a, b .

Appendix G: Regression Methodology

Runtime Model

$$F(n, s, \theta) \approx ans + bn \log n - bn \log s, \text{ where } \theta = \begin{bmatrix} a \\ b \end{bmatrix}$$

- We wish to fit the model above to our data on runtime.
- Our runtime data y consists of (n, s, t_{time}) tuples.

Goal: Minimize Squared Loss

$$C(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} [t_{\text{time},i} - F(n_i, s_i, \theta)]^2$$

- We model this regression problem as a squared-loss minimization problem. Given fixed y , we wish to find the value of θ that minimizes $C(y, \theta)$.
- We use the **Tensorflow** library to perform **gradient descent** along parameter space.

Ideal Initialization Property: Vector has Expected Norm 1

$$\mathbb{E}[|\theta|] = 1$$

- We initialize all weights with the same normal distribution with mean 0— $\mathcal{N}(0, \sigma^2)$.
- Let $X \sim \mathcal{N}(0, \sigma^2)$. Then, notice that:

$$\mathbb{E}[X^2] = \text{Var}(X) + \mathbb{E}[X]^2 = \sigma^2 + 0^2 = \sigma^2$$

- Hence, we have:

$$|\mathbb{E}[\theta]| = \mathbb{E}\left[\sqrt{|\theta|X^2}\right] = \sqrt{|\theta|\mathbb{E}[X^2]} = \sigma\sqrt{|\theta|} = 1$$

$$\sigma = \frac{1}{\sqrt{|\theta|}} = \frac{1}{\sqrt{2}}$$

- Directly running gradient descent on our original model leads to **exploding gradients**. This is most likely caused by an unconstrained output space.
- Hence, taking inspiration from Artificial Neural Networks, we introduced an **activation function** σ :

$$\sigma(x) := \frac{10\sqrt{10}\sqrt[3]{x}}{1 + 10\sqrt{10}|\sqrt[3]{x}|}$$

- Intuitively, σ clamps unconstrained real inputs to $[-1, 1]$. This mitigates large output values, which may lead to exploding gradients during backpropagation.
- We also applied **gradient clipping** to prevent uncontrolled explosions.

- With this activation function, we redefined our cost function as follows:

Goal: Minimize Squared Loss

$$C_{\text{norm}}(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} [\sigma(t_{\text{time},i}) - \sigma(F(n_i, s_i, \theta))]^2$$

- Intuitively, this **smoothens** the cost surface, facilitating gradient descent.
- This smoothing transformation, however, removes the resolution required to hone in on very accurate parameter values θ .

- **Insight:** Use C_{norm} to capture the **high-level geometry** of the cost surface. Use C to capture the **details** of the cost surface.
- This insight motivates the following training procedure:
 - **Find an Approximate Soln:** Run gradient descent for 1000 epochs with C_{norm} .
 - **Refine the Approximation:** Run gradient descent for 1000 epochs with C .
- This algorithm converges with $R^2 = 0.9996$.
- An implementation of this algorithm may be found in `model_regression_analysis.py`.